



UNIVERSITY OF
FRONTIER TECHNOLOGY,
BANGLADESH



 WALTON
PRESENTS
ROBOFUSION
—1.0

TECHATHON ROUND 1 QUESTION SET



**Multi-Hazard Smart Campus
Safety & Response Grid (SCS-RG)**

Organized by: UFTB Robotics Club



SCOPE OF THIS DOCUMENT

This is the Round 1 case. Teams design and build a working prototype (physical hardware or full simulation) and submit source code, system documentation and a video demonstration of the system passing the test cases below. The Grand Finale will use a separate problem statement — this case applies to Round 1 only.

BUILD

• INNOVATE

• SOLVE

Table of Contents

01. Problem Statement.....	3
02. Glossary of Key Terms.....	5
03. System Architecture Overview	7
04. Track Options & Equipment List	9
05. Evaluation Approach	10
06. Section A: Hardware & Sensing [40 marks]	11
07. Section B: Backend System [35 marks]	14
08. Section C: Frontend Command Dashboard [30 marks]	17
09. Section D: Database Design [20 marks].....	19
10. Section E: Integration & Edge Cases [15 marks].....	21
11. Section F: Documentation & Presentation [20 marks]	23
12. Section G: Bonus [up to 40 marks]	25
13. Worked Example: Risk Fusion Formula.....	27
14. Round 1 Submission Procedure.....	29
15. Round 1 Marks Summary.....	30
16. Disclaimer & contact.....	31

01. Problem Statement

Multi-Hazard Smart Campus Safety & Response Grid (SCS-RG)

Design and build a full-stack IoT system that senses multiple hazard types across UFTB's technical labs, fuses them into a risk-ranked live incident feed and gives campus security a command dashboard they can trust during a real, simultaneous, multi-zone emergency.

Target Users: Campus Security Staff, Facilities Admins

Round 1 Total Marks: 200 (160 core, including Documentation & Presentation, + up to 40 Bonus)

The system must:

- ▶ Monitor at least 3 independent zones for fire, gas, water-level/flood and presence/occupancy signals.
- ▶ Compute a per-zone risk score from those signals.
- ▶ Rank zones by response priority when multiple are critical at once, so staff know which one to check first.
- ▶ Run through a real backend, a real database and a real frontend dashboard that security staff would actually trust during a live incident.

Scenario

UFTB's campus runs on its technical labs — the IoT Lab, Robotics Lab, Software Lab, Data Science Lab and the Server Room — but has no unified way to detect and respond to hazards across them. A soldering-related fire risk in the IoT Lab's hardware bench, an overheating GPU rack in the Data Science Lab and an AC condensate leak near the Server Room's cooling unit are all currently handled the same way: someone notices, someone shouts, someone calls someone else. There's no sensing, no automatic response and no way for campus security to know, in real time, which lab needs help first when several things go wrong at once.

Sensor nodes are placed across these representative zones:

Lab / Zone	Realistic Hazard Profile
IoT Lab	Soldering/wiring fire risk, fume/gas risk from flux and adhesives, high occupancy during lab hours
Robotics Lab	Battery off-gassing, fabrication tool fire risk (cutting/welding), moderate occupancy
Server Room	Electrical/overheating fire risk, AC coolant/condensate leak (flood-equivalent), low occupancy but high asset value
Data Science Lab	GPU/server rack overheating, moderate occupancy, minimal gas/flood risk
Software Lab	Primarily occupancy-driven (evacuation headcount), minimal fire/gas/flood risk – a useful "quiet" control zone

Each zone independently senses fire, gas, water-level and occupancy and reports into a backend that decides which zone is most urgent right now, across the whole campus – not just whether any single zone crossed a threshold. Teams choose at least 3 of these five labs to implement.

Teams may build this as physical hardware (Track A) or as a full Tinkercad/Wokwi simulation (Track B) – see Section 04. Both are judged on the same rubric.

02. Glossary of Key Terms

Use these terms the same way throughout your documentation and video – consistent, precise use of this vocabulary makes your submission much easier for judges to follow.

Term	Meaning
Zone	An independently-monitored area (a lab or room) with its own sensors and its own SAFE / WARNING / CRITICAL state.
SAFE / WARNING / CRITICAL	The three-level state of a zone, derived from its risk score. SAFE: no meaningful hazard signal. WARNING: elevated signal, visual notice only. CRITICAL: hazard confirmed – triggers local response and enters the priority queue.
Risk Score	A single number (e.g. 0-100) computed per zone from its raw sensor readings via a weighted formula – not the raw sensor values directly.
Debounce	Requiring a signal to persist for a minimum time or minimum consecutive readings before treating it as real, so a single noise spike doesn't cause a false alarm.
Risk Fusion	Combining a zone's fire, gas, water-level and occupancy signals into that zone's single risk score.
Priority Ranking	The ordering of currently-critical zones by urgency, so staff know which one to respond to first.
Server-Side Computation	Calculating the risk score on the backend from raw sensor inputs, rather than trusting a value reported directly by the zone node.
Race Condition	A bug where the outcome depends on unpredictable timing between two things happening at nearly the same moment.
Referential Integrity	A database property ensuring relationships between tables stay valid – an Incident should never point to a Zone that no longer exists.
RBAC (Role-Based Access Control)	Restricting what a user can do based on their role, enforced by the backend/API – not just by hiding a button in the interface.
Idle State	The system's resting state where all zones are SAFE and no incidents are open.
Edge Case	A scenario outside the simple expected flow – sensor failure, two things happening at once, network or power loss – that the system needs to handle correctly.
Predicted Risk	A machine-learning-derived estimate (bonus feature) of how likely a zone is to become CRITICAL soon, kept separate from the live risk score and never allowed to trigger physical actuation on its own.



1. A zone node samples its sensors on a fixed interval – for example, every 500ms to 1 second is a reasonable starting point.
2. The zone node sends its raw readings to the backend's ingestion endpoint.
3. The backend checks the payload is well-formed, then computes the zone's risk score itself using the formula in Section 13 – the zone node never reports its own risk score or state, only raw numbers.
4. If the computed score crosses a threshold, the zone's state changes. This is saved to the database and pushed out to every connected dashboard.
5. If the new state is CRITICAL, the backend also sends a command back down to that zone node to trigger the buzzer, LED and relay.
6. If more than one zone is CRITICAL at the same time, the priority ranking engine recalculates the ordered list and the dashboard updates to show it.
7. When staff acknowledges an incident from the dashboard, the backend records who acknowledged it and when – only one acknowledgment should actually be recorded if two people try at once.
8. When a zone's risk score drops back below the threshold, the system logs the recovery and that zone returns to idle.

03. System Architecture Overview

3.1 High-Level Block Diagram

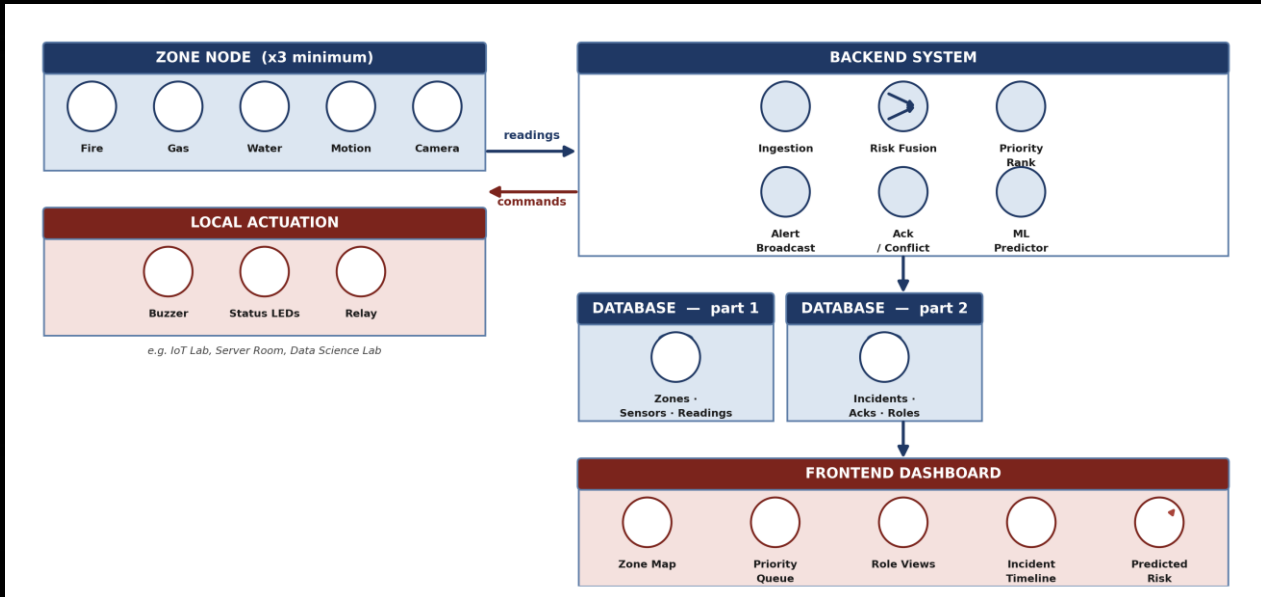


Figure 1 — Zone node, backend, database and frontend responsibilities and data direction.

3.2 Zone Layout

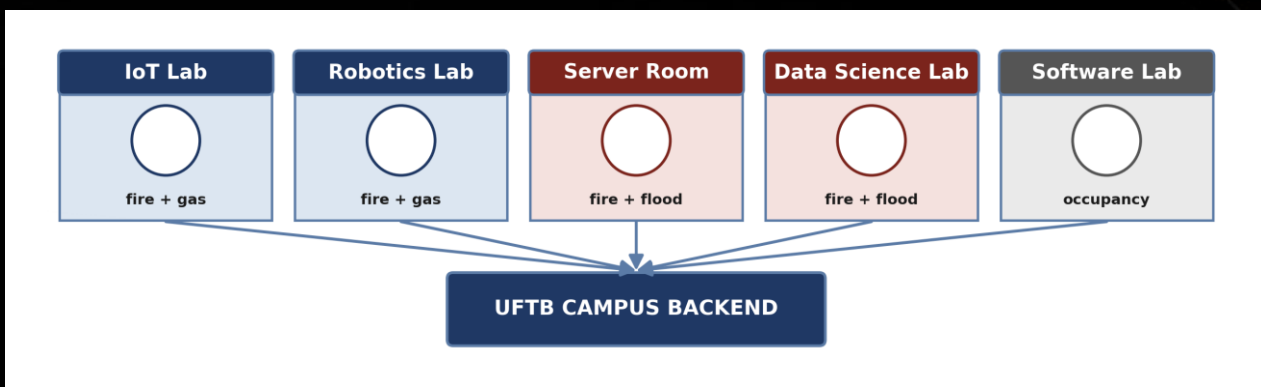


Figure 2 — The five candidate labs, their hazard profile and how each report into the shared campus backend.

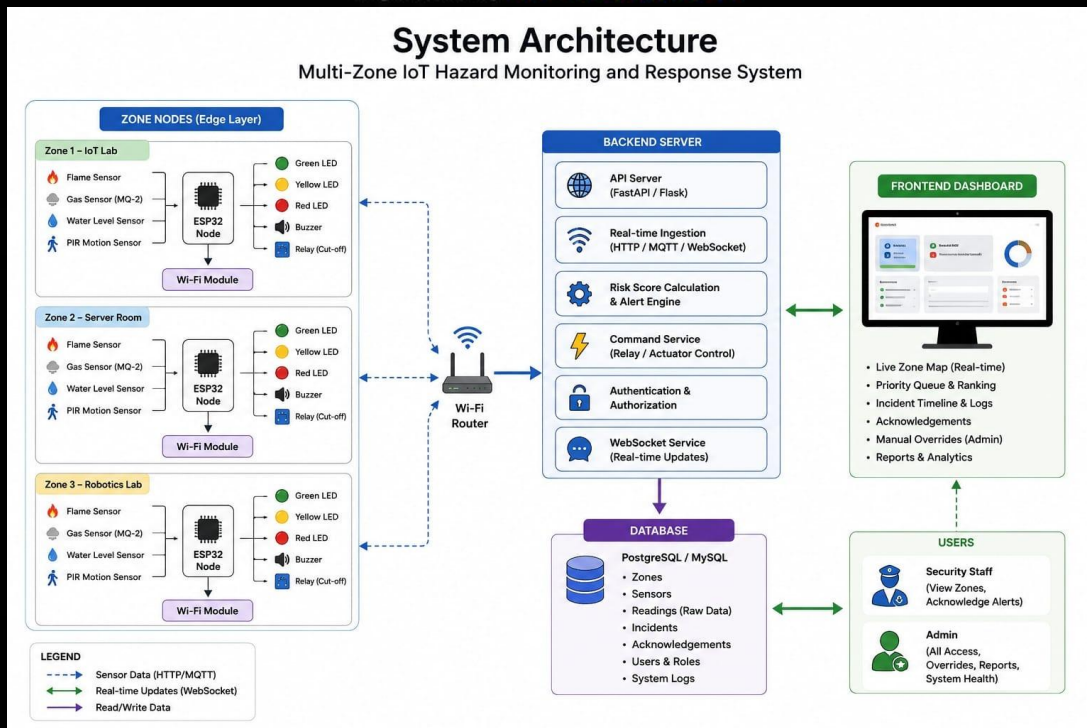


Figure 3 – Overall System Architecture

Pick at least 3 of these 5 labs. A sensible starting set for a small team: IoT Lab (fire + gas + occupancy, the richest single zone), Server Room (fire + flood, high stakes, low occupancy) and Data Science Lab or Robotics Lab as the third, giving two different hazard profiles to prove the fusion and ranking logic actually differentiates between zones rather than treating them identically.

3.3 Data Flow, Step by Step

A good way to prove all of this in your video is to walk through this exact sequence live, narrating each step as it happens on screen.

3.4 What Each Piece Is Responsible For

- ▶ **Zone side:** fire/flame, gas, water-level and PIR sensors; a buzzer; LEDs for green/yellow/red; a relay module standing in for a real power cutoff; Wi-Fi (ESP32) or serial connection to the backend.
- ▶ **Backend:** receives readings, computes risk scores, broadcasts state changes, handles acknowledgments and conflicts, exposes an API and optionally runs the ML predictor.
- ▶ **Database:** stores everything durably – zones, sensors, readings, incidents, acknowledgments and user roles.
- ▶ **Frontend:** the dashboard staff actually look at – live zone map, priority queue, role-based views, incident history and the optional predicted-risk panel.

04. Track Options & Equipment List

Pick one track. Both are judged on the same rubric – Track B is not a lesser option, it's simply a different way to build the same system.

Track A – Physical Hardware

#	Component	Used for
1	ESP32 (or Arduino Uno/Nano) per zone	The zone-node microcontroller
2	Flame/fire sensor	Section 06, Test Case 1
3	MQ-2 or MQ-135 gas sensor	Section 06, Test Case 2
4	Water-level sensor	Section 06, Test Case 3
5	PIR motion sensor	Section 06, Test Case 4
6	ESP32-CAM (only if attempting the camera bonus)	Section 12, Bonus 1
7	Relay module, one per zone	Section 06, Test Case 5
8	Buzzer	Section 06, Test Case 5
9	LEDs (green/yellow/red per zone)	Section 06, Test Case 5
10	Breadboard and jumper wires	Assembly
11	USB cable / power source	Power

If budget or component access is tight, it's fine to build one fully wired zone and simulate the other two zones' sensor inputs in software (e.g. a small script feeding realistic values) – as long as the backend, database and frontend still treat all zones identically and your documentation says clearly which zones are physically wired and which are simulated.

Track B – Simulation (Tinkercad or Wokwi)

Model the same sensor and actuator set using each platform's closest equivalent components. If using Wokwi, its HTTP/WebSocket simulation support should feed your actual backend – the backend, frontend and database are exactly as real as in Track A, only the physical sensing layer is simulated instead of built.

05. Evaluation Approach

Every test case below is checked against your submitted system — via your video demonstration and, where relevant, by reviewing your source code. Marks are earned by your system actually doing what's described, shown clearly in the video with narration explaining what's happening on screen — not by a written description of what it's supposed to do.

Where a test case has several lettered required behaviors, try to show each one as its own short, clearly labeled clip or segment in your video rather than one long unstructured recording — it makes judging faster and fairer and it's also just easier for you to be sure you've covered everything.

06. Section A: Hardware & Sensing [40 marks]

Test Case 1: Fire/Flame Sensor Detection [8 marks]

Conditions: One flame sensor per zone — for example, guarding the IoT Lab or Robotics Lab's soldering station or the Server Room's electrical panel. You decide the detection range and the debounce window.

Required Tests:

- ▶ (a) No flame present → Expected: SAFE
- ▶ (b) A brief flicker shorter than your debounce window → Expected: no trigger
- ▶ (c) A sustained flame longer than your debounce window → Expected: CRITICAL contribution to the risk score
- ▶ (d) The flame source is removed → Expected: the score decays back to SAFE within a recovery window you define

How you might approach this: Sample the sensor every 100-200ms; treat a flame as "real" only after N consecutive HIGH readings (e.g. 5 readings = 1 second); on removal, let the score decay linearly over a few seconds rather than snapping instantly back to zero.

Evaluation: detection accuracy (3), debounce correctness (3), recovery behavior (2).

Test Case 2: Gas Sensor Detection [8 marks]

Conditions: Most relevant in the IoT Lab and Robotics Lab, where soldering fumes and battery off-gassing are realistic.

Required Tests:

- ▶ (a) Baseline clean air → Expected: near-zero contribution
- ▶ (b) Gradually rising concentration → Expected: the contribution rises proportionally, not in a single jump
- ▶ (c) Concentration crosses your CRITICAL threshold → Expected: the zone escalates
- ▶ (d) The first ~30 seconds after power-on (sensor warm-up) → Expected: no false trigger during this period

How you might approach this: Normalize the raw analog reading to a 0.0-1.0 scale based on your sensor's datasheet range, rather than using the raw ADC value directly; ignore readings during the first 30 seconds after boot.

Evaluation: graded response accuracy (3), threshold accuracy (3), warm-up handling (2).

Test Case 3: Water-Level / Flood Sensor Detection [8 marks]

Conditions: Most relevant in the Server Room and Data Science Lab, where an AC condensate or coolant leak is the realistic flood risk.

Required Tests:

- ▶ (a) Dry → Expected: SAFE
- ▶ (b) Rising water level over time → Expected: proportional risk contribution
- ▶ (c) Level crosses your CRITICAL threshold → Expected: the zone escalates
- ▶ (d) The sensor is wetted, then cleared → Expected: correct reset

How you might approach this: Place the sensor at the lowest point under the AC unit or cooling equipment, since that's where a real leak would pool first; test the reset by physically drying the sensor.

Evaluation: accuracy (3), graded response (3), reset correctness (2).

Test Case 4: PIR Presence / Occupancy Detection [8 marks]

Required Tests:

- ▶ (a) Empty zone → Expected: occupancy = false
- ▶ (b) A person enters → Expected: occupancy = true within about 1 second
- ▶ (c) A brief exit and re-entry within ~2 seconds → Expected: this does not spam the log with repeated events
- ▶ (d) Sensor disconnected → Expected: the zone shows OFFLINE, not silently reported as empty

How you might approach this: Most PIR modules already have a built-in retrigger delay you can tune; only log an occupancy change event if the new state holds for at least 1-2 seconds.

Evaluation: detection reliability (3), flicker handling (3), offline detection (2).

Test Case 5: Local Response & Actuation [8 marks]

Conditions: CRITICAL should trigger the buzzer, an LED and a simulated relay cutoff together, within about 1 second. WARNING should only show a visual indicator, with no cutoff.

Required Tests:

- ▶ (a) A zone crosses into CRITICAL → Expected: full response within 1 second, logged with a timestamp

- ▶ (b) A zone is at WARNING only → Expected: no relay or buzzer activation
- ▶ (c) A zone recovers to SAFE → Expected: everything resets, and this is logged
- ▶ (d) Two zones cross into CRITICAL within the same second → Expected: each responds on its own, without interfering with the other

How you might approach this: Be specific about what the relay is "cutting" — for example, simulate cutting power to a lab bench outlet in the IoT Lab or a server rack's non-essential PDU in the Server Room.

Evaluation: timing (3), state gating (3), independence (2).

07. Section B: Backend System [35 marks]

Test Case 6: Real-Time Ingestion & Risk Fusion [11 marks]

Conditions: The risk score must be computed on the backend from raw sensor values – a zone node should never be able to simply report "I am SAFE" and have that trusted directly.

Required Tests:

- ▶ (a) A zone sends raw sensor values → Expected: the backend computes the risk score using the formula documented in Section 13
- ▶ (b) A malformed or out-of-range payload → Expected: rejected with a clear error, not silently absorbed
- ▶ (c) Two hazards cross threshold in the same zone at once → Expected: the combined score reflects both, computed from the real formula
- ▶ (d) The same reading arrives twice (network retry) → Expected: it is not counted twice
- ▶ (e) All zones send readings at their normal interval at once → Expected: none are dropped or noticeably delayed

How you might approach this: A simple sequence number or short time-window de-duplication check catches (d); a lightweight backend framework (Flask, FastAPI, Express) with async handling comfortably handles this for (e).

Evaluation: server-side computation correctness (4), input validation (3), multi-hazard fusion (2), duplicate handling (1), concurrent ingestion (1).

Test Case 7: Alert Broadcast & Conflict Handling [8 marks]

Required Tests:

- ▶ (a) Three zones change state within 1 second of each other → Expected: none of the updates are dropped
- ▶ (b) Two staff members acknowledge the same alert within milliseconds → Expected: exactly one acknowledgment is recorded
- ▶ (c) A zone rapidly flips CRITICAL-SAFE-CRITICAL over a few seconds → Expected: no flood of duplicate incidents
- ▶ (d) An alert already acknowledged and resolved triggers again later → Expected: this creates a new incident record

How you might approach this: For (b), a database unique constraint or a simple "first write wins" check (only update the acknowledgment field if it's currently null) is enough.

Evaluation: concurrent update handling (2), race-condition safety (3), backend-level debounce (2), incident lifecycle correctness (1).

Test Case 8: API Design [6 marks]

Conditions: A documented REST or WebSocket API covering, at minimum: reading current zone status, querying historical incidents, acknowledging an alert and an admin manual override.

Required Tests:

- ▶ (a) The zone-status endpoint returns the current state of all zones in a single call
- ▶ (b) The historical-incident endpoint supports filtering by at least a date range
- ▶ (c) The acknowledgment endpoint rejects a request for an incident ID that doesn't exist

How you might approach this: A small table in your documentation — method, path, one-line purpose, example JSON — is enough.

Evaluation: completeness (3), sensible design with example payloads (2), correct handling of the three request patterns (1).

Test Case 9: Backend Resilience [4 marks]

Required Tests:

- ▶ (a) The backend process is stopped mid-incident and restarted → Expected: it re-constructs current zone states from the database
- ▶ (b) A zone node loses network connectivity → Expected: it caches readings locally and resyncs once reconnected

How you might approach this: On startup, have the backend query the most recent reading and incident state for each zone before it starts accepting new connections.

Evaluation: state recovery (3), offline caching (1).

Test Case 10: Security & Authentication [3 marks]

Required Tests:

- ▶ (a) A reading sent from a source that isn't a registered zone → Expected: rejected
- ▶ (b) A dashboard API call made without a valid session or token → Expected: rejected

How you might approach this: A simple per-zone API key sent in a header, checked against a small table of registered zones, is entirely sufficient.

Evaluation: zone authentication (2), dashboard session/token authentication (1).

Test Case 11: Scalability & Load Handling [3 marks]

Required Tests:

- ▶ (a) Simulate load from significantly more zones than your physical minimum of 3
→ Expected: backend and dashboard stay responsive
- ▶ (b) Briefly describe what would need to change to run this across a real campus with 30+ zones

How you might approach this: A short script that fires simulated readings for extra "phantom" zones alongside your real ones is an easy way to demonstrate (a).

Evaluation: demonstrated load handling (2), scaling reasoning (1).

08. Section C: Frontend Command Dashboard [30 marks]

Test Case 12: Live Zone Map & Priority Queue [12 marks]

Required Tests:

- ▶ (a) The zone map's colors update live, without needing a manual page refresh
- ▶ (b) When two or more zones are CRITICAL at once, a priority queue view ranks them by risk score, occupancy and time critical
- ▶ (c) The ranking shows why each zone is ranked where it is
- ▶ (d) Someone looking at the screen for the first time should identify the most urgent zone within about 2 seconds

How you might approach this: A WebSocket connection pushing state changes is cleanest for (a); a 1-2 second polling interval is an acceptable fallback if you're short on time.

Evaluation: real-time update mechanism (4), ranking correctness (4), visible justification (4).

Test Case 13: Role-Based Access Control [6 marks]

Conditions: "Security staff" can view zones and acknowledge alerts. "Admin" can additionally issue manual overrides and view system health.

Required Tests:

- ▶ (a) A staff-role user tries an admin-only action through the interface → Expected: blocked
- ▶ (b) A staff-role user tries the same action through a direct API call, bypassing the interface → Expected: also blocked

How you might approach this: Check the user's role on the backend at the start of every admin-only endpoint handler, not just in the frontend's routing logic.

Evaluation: interface-level restriction (2), backend enforcement (4).

Test Case 14: Incident Timeline & Reporting [5 marks]

Required Tests:

- ▶ (a) A searchable, filterable log of past incidents — zone, hazard type, duration, who acknowledged, when it resolved

- ▶ (b) A completed incident's full timeline, from first trigger through acknowledgment to recovery

How you might approach this: Storing each state transition as its own row (rather than overwriting one "current state" field) makes this timeline nearly free to build.

Evaluation: completeness (3), usability (2).

Test Case 15: Notification & Alert UX [4 marks]

Required Tests:

- ▶ (a) When a zone first goes CRITICAL, the dashboard draws attention to it beyond just changing a color
- ▶ (b) Several alerts arriving close together are all noticeable, not lost in a single generic notification
- ▶ (c) Once an alert is acknowledged, it stops actively demanding attention

How you might approach this: A small toast/banner component that stacks when multiple alerts arrive, paired with a short audio cue, is usually enough.

Evaluation: attention-drawing mechanism (2), multi-alert handling (1), notification-clearing behavior (1).

Test Case 16: Dashboard Usability Under Mixed Conditions

[3 marks]

Required Tests:

- ▶ (a) With zones in a realistic mix of states at once, the dashboard stays readable rather than cluttered
- ▶ (b) State one accessibility consideration in your design (e.g. not relying on color alone)

How you might approach this: Pairing each status color with a distinct icon or label satisfies (b) simply and makes the dashboard clearer for everyone.

Evaluation: demonstrated mixed-state clarity (2), accessibility consideration (1).

09. Section D: Database Design [20 marks]

Test Case 17: Schema Design [7 marks]

Conditions: A schema with clearly related tables for Zones, Sensors, Readings, Incidents, Acknowledgments and Users_Roles — not one large table holding everything. State primary and foreign keys explicitly, e.g. Readings.zone_id -> Zones.id.

How you might approach this: Zones describes what exists, Readings is the raw sensor history, Incidents is the derived "something happened" record, and Acknowledgments/Users_Roles handle who did what — keeping these separate lets you answer both "what did the sensor read" and "what did the system decide".

Evaluation: normalization and relationship correctness (4), explicit key design (3).

Test Case 18: Concurrency & Integrity [5 marks]

Required Tests:

- ▶ (a) Ten simultaneous sensor writes across different zones → Expected: no data loss
- ▶ (b) Attempting to delete a zone that still has open incidents → Expected: blocked or archived
- ▶ (c) A reading arrives with a timestamp earlier than the zone's last recorded reading → Expected: doesn't scramble the timeline

How you might approach this: For (b) a foreign key constraint with ON DELETE RESTRICT (or an explicit archive flag instead of a real delete) is a one-line fix.

Evaluation: concurrent write safety (2), integrity constraint demonstrated (2), timestamp handling (1).

Test Case 19: Query Performance [4 marks]

Conditions: The query "all CRITICAL incidents in the last 24 hours across all zones" returns quickly even against a larger seeded dataset (e.g. 10,000+ reading rows). State which column(s) you indexed and why.

How you might approach this: An index on the Incidents table's (status, created_at) columns together is usually the single biggest win for this query.

Evaluation: query and indexing design (3), demonstrated reasoning (1).

Test Case 20: Backup & Recovery Strategy [3 marks]

Required Tests:

- ▶ (a) A stated backup approach — even a simple scheduled export is fine
- ▶ (b) A described (or demonstrated) recovery path: what happens if the database is lost and what would be lost in the gap

How you might approach this: A daily pg_dump / sqlite3 .backup (or equivalent) written to a separate file location is a perfectly reasonable answer.

Evaluation: backup strategy stated (2), recovery path described (1).

Test Case 21: Data Retention & Access [1 mark]

Conditions: A stated policy for how long historical readings and incidents are kept and who can see raw historical data versus just summary views, tying back to the roles from Test Case 13.

How you might approach this: "Raw readings older than 90 days are summarized and dropped; only admins can query raw historical readings" is a complete, reasonable answer.

Evaluation: retention/access policy stated and reasoned (1).

10. Section E: Integration & Edge Cases [15 marks]

Test Case 22: End-to-End Multi-Zone Incident [6 mark]

Show this as one continuous demonstration:

1. Two zones cross into CRITICAL within seconds of each other
2. The backend computes both risk scores and ranks the two zones
3. The dashboard shows both, with the priority queue correctly ordered and explained
4. Staff acknowledges the higher-priority zone first
5. Both incidents are logged correctly, each with a full timeline
6. Both zones recover to SAFE and the system returns to its idle state

Evaluation: flow without errors (2), correct priority ordering (2), correct logging (1), correct return to idle (1).

Test Case 23: Edge-Case Handling [6 mark]

Show at least three of the following working correctly — more is stronger, but three is the expected minimum:

- ▶ (a) A sensor is disconnected mid-incident → the zone shows OFFLINE, not a false SAFE
- ▶ (b) Three or more zones go CRITICAL within the same second → the ranking still comes out in a consistent, explainable order
- ▶ (c) An admin's manual override happens at the same instant as a real sensor trigger on the same zone → no double response, log clearly shows which caused the action
- ▶ (d) A dashboard browser disconnects and reconnects mid-incident → it catches up to the current state rather than showing stale data
- ▶ (e) The whole system loses power and restarts → it rebuilds zone states from the database rather than assuming everything is SAFE
- ▶ (f) A sensor reports an impossible value, like a negative water level → it's rejected or flagged rather than fed straight into the risk formula

How you might approach this: Pick the 3-4 scenarios that are easiest for your specific build to demonstrate convincingly and do those well, rather than attempting all six shallowly.

Evaluation: up to 6 marks distributed across the scenarios shown, by how correctly and clearly each is demonstrated.

Test Case 24: Behavior Under Combined Load [2 mark]

Conditions: With multiple zones active at once and at least one of them cycling quickly through SAFE-WARNING-CRITICAL-SAFE, the system keeps running normally – no crash, no freeze, no dashboard showing obviously wrong information.

Evaluation: demonstrated stability (2).

Test Case 25: Cross-Component Consistency [1 mark]

Conditions: At any moment you're asked to pause during your demonstration, the zone node's own LED, the backend's stored state for that zone and what the dashboard is displaying should all agree with each other.

How you might approach this: This is really a proof that your system doesn't secretly track state in two places that can drift apart – if the dashboard always reads its state from the same backend/database source of truth, this naturally holds.

Evaluation: demonstrated three-way consistency (1).

11. Section F: Documentation & Presentation [20 marks]

This section is scored in its own right — strong engineering that is poorly documented or poorly demonstrated loses real marks here, separately from the sections above.

Test Case 26: Circuit Diagrams [3 mark]

Required: a circuit diagram for each implemented zone — sensor wiring, pin configuration, and power supply, clear enough that someone else could rebuild your zone node from it alone.

Evaluation: completeness and correctness of wiring shown (2), clarity/labeling (1).

Test Case 27: Software Architecture Diagram [3 mark]

Required: a diagram matching the shape of Section 03 — zone node, backend, database, and frontend, with the direction of data flow marked.

Evaluation: accuracy against your actual build (2), clarity (1).

Test Case 28: API Documentation [3 mark]

Required: the endpoints from Test Case 8, documented with method, path and at least one example request/response.

Evaluation: completeness (2), example payload quality (1).

Test Case 29: Database Schema Diagram [3 mark]

Required: the schema from Test Case 17, shown as an actual entity-relationship diagram, not just a bullet list of table names.

Evaluation: accuracy against your actual schema (2), diagram clarity (1).

Test Case 30: Risk Fusion Formula & Justification [3 mark]

Required: your actual weighted risk formula (see Section 13), with the weights explained in your own reasoning, not left as unexplained numbers.

Evaluation: correctness/consistency with your build (2), quality of justification (1).

Test Case 31: Video Demonstration Quality [5 marks]

Required: a video, maximum 7 minutes, showing the test cases from Sections 06-10 passing, narrated clearly enough that a judge who has never seen your system can follow what's happening and why.

- ▶ Your working prototype, shown clearly on screen
- ▶ The test cases from Sections 06-10 being executed, each identifiable as its own segment
- ▶ A narrated walkthrough explaining what's happening as it happens
- ▶ Any bonus tasks you attempted

Evaluation: organization and pacing within the 7-minute limit (2), narration clarity (3) — going over 7 minutes does not disqualify the submission, but content beyond the limit will not be reviewed.

12. Section G: Bonus [up to 40 marks]

These are optional and additive — attempting none of them does not cost you anything from the 160 core marks above.

Bonus 1: Camera-Based Occupancy Check [10 marks]

Use an ESP32-CAM to get a rough sense of whether a zone is occupied, cross-checked against the PIR sensor, to reduce false "zone is empty" readings that would otherwise skew the priority ranking. Simple motion or frame-difference detection is enough.

Evaluation: correctness of the cross-check against PIR (6), integration into the priority ranking (4).

Bonus 2: Short-Term Risk Trend [10 marks]

Track whether a zone's risk score has been rising over its last several readings, and flag it as "trending toward CRITICAL" before it actually crosses the threshold. A simple moving average or slope calculation over a short window is sufficient.

Evaluation: correctness of the trend detection (6), clear dashboard presentation of the trend (4).

Bonus 3: Machine-Learning Risk Prediction [10 marks]

Train an actual machine learning model — not just a hardcoded rule and not just Bonus 2's simple trend — that estimates the probability of a zone reaching CRITICAL within the next few minutes, based on its recent sensor history.

What to include:

- ▶ (a) What data you trained it on — synthetic/simulated data is fine, but say so clearly
- ▶ (b) What kind of model you used and why — a simple logistic regression or small time-series model is a completely reasonable choice
- ▶ (c) How you validated it — an accuracy, precision/recall or similar metric, actually reported
- ▶ (d) How the prediction appears on the dashboard as its own clearly separate "Predicted Risk" indicator, never mixed with the live risk score
- ▶ (e) A clear statement that this predicted value can never, by itself, trigger the relay cutoff or buzzer

Evaluation: soundness and honesty of the approach (6), correct dashboard integration (2), the safety statement in (e) actually being respected by the design (2).

Bonus 4: Natural-Language Incident Reporting [10 marks]

Let a staff member type a short free-text note — for example, "smell of gas near the IoT Lab bench, not sure how bad" — and have a language-model or similar reasoning layer turn that into a structured signal (zone, hazard type, estimated severity) that feeds into the priority ranking, along with a reply confirming what it understood. Whatever this layer produces must pass through the same validation checks as any other input before it's allowed to affect the ranking — it should never be able to act on its own unchecked output.

Evaluation: correctness of the free-text-to-structured-signal conversion (5), the deterministic validation gate actually being enforced before the ranking is affected (5).

13. Worked Example: Risk Fusion Formula

This shows the kind of specificity your own formula should have – adapt or replace the actual numbers with your own reasoning.

```
risk_score(zone) =
    w_fire    * fire_signal (0 or 1, after debounce)
+ w_gas      * gas_level_norm (0.0 to 1.0, normalized to sensor range)
+ w_water    * water_level_norm (0.0 to 1.0)
+ w_occ      * occupancy_factor (0.0 to 1.0, higher if the zone is occupied)
```

where: w_fire = 40, w_gas = 25, w_water = 20, w_occ = 15
(these weights add up to 100, so risk_score naturally falls in 0-100)

Classification:

risk_score < 30	-> SAFE
30 <= risk_score < 65	-> WARNING
risk_score >= 65	-> CRITICAL

Here, fire is weighted highest because it's the fastest-escalating and most damaging hazard in a lab full of electronics. Occupancy is weighted lowest on its own, since an empty zone with a real hazard is still dangerous to responders and equipment – but occupancy matters more when ranking between zones (Section 08, Test Case 12) than it does for a single zone's own score. Use whatever weights make sense for your own reasoning, as long as you explain them.

14. Round 1 Submission Procedure

Deadline: 27 July 2026, 12:00 AM (Bangladesh Standard Time). The case is released 25 July 2026, 12:00 AM.

1. Source code, as a public GitHub repository

Do not push any changes to this repository after you submit it. Include:

- ▶ Zone-node code (Arduino/ESP32) or your simulation project files (Tinkercad/Wokwi)
- ▶ Backend server code
- ▶ Frontend dashboard code
- ▶ Database schema/migration files
- ▶ A README with setup instructions

2. System documentation, as a PDF

Covering everything listed in Section 11 — circuit diagrams, architecture diagram, API documentation, database schema and your risk fusion formula with its weights explained.

3. A video demonstration, maximum 7 minutes

Upload it to Google Drive and share it with general viewer access. It should show:

- ▶ Your working prototype
- ▶ The test cases from Sections 06-10 being executed
- ▶ A narrated walkthrough of the system
- ▶ Any bonus tasks you attempted

Content beyond the 7-minute mark will not be reviewed by judges — plan your narration and pacing accordingly (see Test Case 31).

File naming: RoboFusion_[SegmentName]_[TeamName]_R1

Corrupted, incorrectly formatted or late files may result in immediate disqualification.

The top qualifying teams advance to the Grand Finale, held on-site at UFTB Campus on 08 August 2026 (finalists announced 28 July 2026). The Grand Finale problem will be a separate, different case.



15. ROUND 1

MARKS SUMMARY

SECTION	MARKS
A. HARDWARE & SENSING	40
B. BACKEND SYSTEM	35
C. FRONTEND DASHBOARD	30
D. DATABASE DESIGN	20
E. INTEGRATION & EDGE CASES	15
F. DOCUMENTATION & PRESENTATION	20
CORE TOTAL	160

	BONUS 1: CAMERA-BASED OCCUPANCY	+10
	BONUS 2: SHORT-TERM RISK TREND	+10
	BONUS 3: ML-BASED RISK PREDICTION	+10
	BONUS 4: NATURAL-LANGUAGE INCIDENT REPORTING	+10
BONUS TOTAL		+40
ROUND 1 GRAND TOTAL (MAX)		200



UNIVERSITY OF
FRONTIER TECHNOLOGY,
BANGLADESH



TECHATHON ROUND 1 QUESTION SET



**Multi-Hazard Smart Campus
Safety & Response Grid (SCS-RG)**

DISCLAIMER



The organizing committee reserves the right to change any rule without prior notice and may disqualify or ban any team for plagiarism or pre-made projects, sharing confidential case materials with non-participants, sub-mitting corrupted or late files, misconduct, false registration information or tampering with other teams' equipment or submissions.

Judges' decisions are final in case of disputes.

CONTACT



Mumith Chowdhury

✉ mumith0001@std.uftb.ac.bd
☎ 01616093491



Ahmed Shahariar Uday

✉ shahariar0001@std.uftb.ac.bd
☎ 01766228175

Contact for any kind of information

Best of luck!

BUILD

• INNOVATE

• SOLVE